WT Experiment 4

SR NO:	CLASS	NAME	ROLL NO
1.	D15A	Siddhant Sathe	50
2.	D15A	Arnav Sawant	52
3.	D15A	Pranav Titambe	60

AIM: Component and Arduino Introduction

Component and Arduino Introduction

The Arduino Uno is a versatile and widely used microcontroller board that acts as the heart of many electronics projects. It is an open-source platform that allows users to design and develop interactive electronic devices. The board consists of an Atmega328 microcontroller that can be programmed to control various hardware components, making it ideal for a variety of applications in IoT, robotics, home automation, and more.

In this experiment, we will explore how to use an Arduino Uno to display real-time temperature readings from a TMP36 temperature sensor on a 16x2 LCD display. The main goal is to read the analog temperature data, convert it into a human-readable temperature format, and display it on the LCD screen.

Arduino Uno Overview

The **Arduino Uno** is one of the most popular boards in the Arduino family due to its simplicity, low cost, and ease of use. It features:

- **Microcontroller**: Atmega328P
- **Operating Voltage**: 5V
- **Digital I/O Pins**: 14 (of which 6 can be used as PWM outputs)
- **Analog Input Pins**: 6
- **Flash Memory**: 32 KB
- **Clock Speed**: 16 MHz
- **USB Connection**: For programming and communication
- **Power Supply**: Can be powered via USB or an external power supply

The Arduino Uno can be programmed using the Arduino IDE, which allows users to write and upload code that will control various electronic components connected to the board.

Components Used in This Experiment

1. **Node MCU ESP 8266**:
   - Acts as the central controller that processes the sensor data and drives the LCD.
2. **RFID Sensor**:
   - An RFID sensor (Radio-Frequency Identification sensor) is a device that uses electromagnetic fields to automatically identify and track objects. It works by detecting signals from RFID tags, which are small devices containing a chip and an antenna. These tags store

information and transmit it wirelessly when activated by the RFID sensor. RFID sensors are commonly used in inventory management, access control, transportation systems, and even pet identification. They're fast, efficient, and don't require direct line-of-sight to read the tags..

3. **16x4 LCD Display**:
    - A liquid crystal display used to show the temperature readings. It has 16 characters in each of its 4 rows and can be used to display messages or numbers.
    - The `LiquidCrystal` library is used to control the display and print data onto it.
4. **Jumper Wires**:
    - Used to connect the components (Arduino, sensor, and LCD) together on a breadboard or circuit.
5. **Breadboard**:
    - A tool for creating prototype circuits without soldering. It provides a simple way to build the connections between components.

Code Explanation

The code uses an RFID module (MFRC522) and an ESP8266 WiFi module to:

1. Scan RFID cards/tags to read data.
2. Display a greeting on an LCD screen.
3. Send the cardholder's name to a Google Sheet using an HTTP request.

```
#include <SPI.h>
#include <MFRC522.h>
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
#include <WiFiClientSecureBearSSL.h>
#include <LiquidCrystal_I2C.h>
#define RST_PIN  D3
#define SS_PIN   D4
#define BUZZER   D8

MFRC522 mfrc522(SS_PIN, RST_PIN);
MFRC522::MIFARE_Key key;
MFRC522::StatusCode status;


int blockNum = 2;
byte bufferLen = 18;
byte readBlockData[18];

String card_holder_name;
const String sheet_url =
"https://script.google.com/macros/s/AKfycbzoyb0VXBUY8lcp4pCJ-4x_Q2dCXOlnfds7
otEZUDt4EkmQeTUriMAoRWSt1IMiqxcu/exec?name=";  //Enter Google Script URL

#define WIFI_SSID "Ghe hotspot"  //Enter WiFi Name
#define WIFI_PASSWORD "00000000"  //Enter WiFi Password

//Initialize the LCD display
LiquidCrystal_I2C lcd(0x27, 16, 4);
```

```
void setup()
{
    /* Initialize serial communications with the PC */
  Serial.begin(9600);
  //Serial.setDebugOutput(true);

  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("  Initializing  ");
  for (int a = 5; a <= 10; a++) {
    lcd.setCursor(a, 1);
    lcd.print(".");
    delay(500);
  }

   //WiFi Connectivity
  Serial.println();
  Serial.print("Connecting to AP");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED){
    Serial.print(".");
    delay(200);
  }
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  Serial.println();
    /* Set BUZZER as OUTPUT */
  pinMode(BUZZER, OUTPUT);
    /* Initialize SPI bus */
  SPI.begin();

}
 void loop()
{
    lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(" Scan your Card ");
  /* Initialize MFRC522 Module */
  mfrc522.PCD_Init();
  /* Look for new cards */
  /* Reset the loop if no new card is present on RC522 Reader */
  if ( ! mfrc522.PICC_IsNewCardPresent()) {return;}
  /* Select one of the cards */
  if ( ! mfrc522.PICC_ReadCardSerial()) {return;}
  /* Read data from the same block */
  //--------------------------------------------------
  Serial.println();
  Serial.println(F("Reading last data from RFID..."));
```

```
  ReadDataFromBlock(blockNum, readBlockData);
  Serial.println();
  Serial.print(F("Last data in RFID:"));
  Serial.print(blockNum);
  Serial.print(F(" --> "));
  for (int j=0 ; j<16 ; j++)
  {
    Serial.write(readBlockData[j]);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Hey " + String((char*)readBlockData) + "!");
  }
  Serial.println();
  digitalWrite(BUZZER, HIGH);
  delay(200);
  digitalWrite(BUZZER, LOW);
  delay(200);
  digitalWrite(BUZZER, HIGH);
  delay(200);
  digitalWrite(BUZZER, LOW);
  if (WiFi.status() == WL_CONNECTED) {
    std::unique_ptr<BearSSL::WiFiClientSecure>client(new
BearSSL::WiFiClientSecure);
        client->setInsecure();
    card_holder_name = sheet_url + String((char*)readBlockData);
    card_holder_name.trim();
    Serial.println(card_holder_name);

    HTTPClient https;
    Serial.print(F("[HTTPS] begin...\n"));
           if (https.begin(*client, (String)card_holder_name)){

      // HTTP
      Serial.print(F("[HTTPS] GET...\n"));
      // start connection and send HTTP header
      int httpCode = https.GET();

      // httpCode will be negative on error
      if (httpCode > 0) {
        // HTTP header has been send and Server response header has been
handled
        Serial.printf("[HTTPS] GET... code: %d\n", httpCode);
        // file found at server
    lcd.setCursor(0, 1);
    lcd.print(" Data Recorded ");
    delay(2000);
      }

      else
      {Serial.printf("[HTTPS] GET... failed, error: %s\n",
https.errorToString(httpCode).c_str());}

      https.end();
      delay(1000);
```

```
    }
    else {
      Serial.printf("[HTTPS} Unable to connect\n");
    }
void ReadDataFromBlock(int blockNum, byte readBlockData[])
{
    for (byte i = 0; i < 6; i++) {
    key.keyByte[i] = 0xFF;
  }

  /* Authenticating the desired data block for Read access using Key A */
  status = mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A,
blockNum, &key, &(mfrc522.uid));

  if (status != MFRC522::STATUS_OK){
     Serial.print("Authentication failed for Read: ");
     Serial.println(mfrc522.GetStatusCodeName(status));
     return;
  }
    else {
    Serial.println("Authentication success");
  }
    /* Reading data from the Block */
  status = mfrc522.MIFARE_Read(blockNum, readBlockData, &bufferLen);
  if (status != MFRC522::STATUS_OK) {
    Serial.print("Reading failed: ");
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }

  else {
    Serial.println("Block was read successfully");
  }
  }
```

Code Breakdown

Libraries:

- `MFRC522.h` – For handling RFID communication.
- `ESP8266WiFi.h` and `ESP8266HTTPClient.h` – For WiFi and HTTP requests.
- `LiquidCrystal_I2C.h` – For LCD control.

Constants and Pins:

- `RST_PIN` and `SS_PIN` are connected to the RFID reader.
- `BUZZER` is used for sound notifications.
- WiFi credentials are defined (`WIFI_SSID`, `WIFI_PASSWORD`).

Objects:

- `mfrc522`: Interface with the RFID module.
- `lcd`: Controls the LCD display.

Loop Function

- The `loop()` continuously checks for an RFID card scan.
- Upon detecting a card:
    1. Reads data from a specific block of the RFID tag.
    2. Displays a personalized message on the LCD.
    3. Sends the cardholder's name to a Google Sheet using a secure HTTP request.

Google Sheet Integration

- The URL is preconfigured to interact with a Google Apps Script.
- The RFID tag data (`readBlockData`) is appended to the URL as the `name` parameter.

LCD Output

- When a card is scanned, the LCD screen:
    - Displays a greeting with the cardholder's name (`Hey John!`).
    - Shows "Data Recorded" if the HTTP request succeeds.

Conclusion

This experiment demonstrates how to use the Node MCU ESP 8266 to interface with a RFID sensor and an LCD display. By combining the Node MCU's processing capabilities with simple hardware components like the RFID sensor and the LCD, you can easily create a real-time Smart attendance system. This is a fundamental experiment for understanding analog-to-digital conversion, sensor interfacing, and display output on embedded systems.